

Quiz de abertura: o que cada linha imprime?

Anote o seu palpite para cada trecho **antes** de executar. Vamos conferir ao longo da aula.

1 · Python

```
print(-2 ** 2, 2 ** 3 ** 2)
```

2 · Java

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

3 · JavaScript

```
console.log("5" - 2, "5" + 2);
```

4 · C

```
printf("%d\n", 3 > 2 > 1);
```

5 · Python

```
x = 0
print(x or "padrão", 3 > 2 > 1)
```

6 · JavaScript

```
console.log(0 == "", "" == "0", 0 == "0");
```

As respostas aparecem ao longo da aula · Sebesta Cap. 7

Python no geral deve ser escrito da melhor forma respeitando os termos matemáticos utilizando de parênteses, já o Java tem erro se colocar uma String mesmo sendo número em aspas, neste caso ambos persistem em erro.

Expressões aritméticas: questões de projeto

- Quais são as regras de **precedência**?
- Quais são as regras de **associatividade**?
- Em que **ordem os operandos** são avaliados?
- Há restrições a **efeitos colaterais**?
- O usuário pode **sobrecarregar** operadores?
- Que **mistura de tipos** é permitida?

Lisp dispensa a precedência (p. 306)

(+ a (* b c)): notação prefixa, e os parênteses dizem a ordem.

$a + b * c$ com $a=3, b=4, c=5$

da esquerda para a direita

$(3 + 4) * 5 = 35$

da direita para a esquerda

$3 + (4 * 5) = 23$

com precedência (* antes de +)

$3 + 4 * 5 = 23$

APL: sempre da direita para a esquerda (p. 304)

$3 \times 4 + 5 = 27$

As linguagens possuem diferenças entre formas de leitura tendo peculiaridades em ordem que serão lidos, algumas restrições ou peculiaridades para a leitura do código o mesmo serve para contas matemáticas parênteses e potência.

Precedência e associatividade na prática

Expressão	Python	JavaScript	Java
-2 ** 2	-4	erro de sintaxe	(não há **)
2 ** 3 ** 2	512	512	—
1 + 2 + "3"	TypeError	"33"	"33"
-7 / 2 (inteiros)	-4 (com //)	-3.5	-3
-7 % 2	1	-1	-1

Associatividade (p. 303)

Esquerda: * / % + -
Direita: ** (Python, JS, Ruby, Fortran),
++, --, menos unário
Visual Basic: ^ à esquerda!

Excel

= -2^2 dá 4: lá o menos unário vence o
^.

Na dúvida, **use parênteses** (p. 305): eles deixam a intenção explícita para o compilador e para o próximo leitor.

Sebesta, 11ª ed. · §7.2.1.1-7.2.1.3 · p. 301-305 · exemplos/01_precedencia

Como é mencionado pelo slide na dúvida utilize parênteses com eles a intenção será deixada clara para o compilador e a leitura tem mais chances de sair corretamente.

Ordem dos operandos e efeitos colaterais

Exemplo do livro (p. 307)

```
int a = 5;

int fun1(void) {
    a = 17;      /* efeito colateral */
    return 3;
}

int main(void) {
    a = a + fun1(); /* 8 ou 20? */
    printf("a = %d\n", a);
}
```

8

Java

a ordem é garantida: da esquerda para a direita (p. 308)

20

C (gcc 14)

a ordem não é definida pelo padrão

20

Go

a especificação não define a leitura de a em relação à chamada

Transparência referencial (p. 308): sem efeitos colaterais, a ordem deixa de importar, e $f(x) + f(x)$ pode virar $2 * f(x)$.

Sebesta, 11ª ed. · §7.2.2 · p. 307-309 · exemplos/02_efeitos_colaterais

Falando e peculiaridades do Java ele segue a leitura da esquerda para a direita, já o c não tem uma definição padrão, seguimos um padrão de referencial para escrita.

Operadores sobrecarregados

- O mesmo símbolo com **vários significados**: + soma inteiros, reais e concatena (Java)
- Risco (p. 309): & em C++ é E bit a bit e "endereço de"
- **Definida pelo usuário** (p. 404): C++, C#, Python, Rust, Kotlin. Java **não** permite

Java

```
System.out.println('a' + 1); // 98
System.out.println(" " + 'a' + 1); // a1
x.add(y).multiply(tres); // BigDecimal: sem +
```

Python · sobrecarga com métodos especiais

```
class Vetor:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, outro): # v1 + v2
        return Vetor(self.x + outro.x,
                      self.y + outro.y)

    def __mul__(self, k): # v * 3
        return Vetor(self.x * k, self.y * k)

v = Vetor(1, 2) + Vetor(3, 4) # Vetor(4, 6)
print(v * 3) # Vetor(12, 18)
```

Sebesta, 11ª ed. · §7.3 · p. 309-310 e §9.11 · p. 404 · exemplos/03_sobrecarga

Este código possui um grande problema de legibilidade onde não é possível se situar da parte que está sendo lida prejudicando tanto leitura quanto manutenção do código.

Conversões de tipos: alargamento, estreitamento e coerção

Java · coerção e cast

```
int grande = 123456789;
float f = grande; // alargamento implícito
System.out.println((int) f); // 123456792 !

double d = 3.99;
System.out.println((int) d); // 3 (cast)
System.out.println((byte) 200); // -56

byte b = 10, c = 20;
// byte s = b + c; // erro: b + c é int (p. 312)

System.out.println(1 / 2); // 0
System.out.println(1 / 2.0); // 0.5
```

Linguagem	int → real (implícito)	int32 + int64
C	sim	coerção
Java	sim (alargamento)	coerção para long
Python	sim	(um só tipo int)
Go	não : float64(i)	erro de compilação
Rust	não : i as f64	erro de compilação
ML, F# (p. 312)	não	erro de compilação

Coerção esconde erros (p. 312): em `d = b * a`; com `a` digitado por engano no lugar de `c`, o Java converte e compila.

Algumas linguagens não recebem de forma totalmente segura essa conversão de tipo, podendo perder precisão, podendo converter para um tipo que não suporta os valores corretamente.

Coerção implícita: JavaScript e PHP

JavaScript

```
"5" - 2           // 3
"5" + 2           // "52"
"5" * "2"         // 10
true + 1          // 2
[] + {}           // "[object Object]"
null + 1          // 1
undefined + 1     // NaN
"b" + "a" + "a" + "a" // "baNaNa"

0 == ""          // true
"" == "0"        // false
0 == "0"         // true  (não é transitivo!)
0 === "0"        // false (=== não converte)
```

PHP - "type juggling"

```
var_dump("0e123" == "0e456"); // true !
var_dump("0e123" === "0e456"); // false
var_dump("1" == "01"); // true

md5("240610708") // "0e462097431906..."
md5("QNKCDZO")   // "0e830400451993..."
// == entre os dois hashes: true
```

Regra prática

Em JavaScript e PHP, compare com `===` e `!==`, que **não** fazem coerção.

O uso indevido de `+`, `==` para fazer comparação e concatenação de forma equivocada, para conseguir obter um resultado.

Expressões relacionais e booleanas

C - booleano é inteiro (p. 315)

```
int a = 3, b = 2, c = 1;
printf("%d\n", a > b > c); // 0 !
// (3 > 2) vale 1, e 1 > 1 é falso

printf("%d\n", (a > b) && (b > c)); // 1

if (-1) printf("verdadeiro\n"); // != 0
```

Python - comparações encadeadas

```
a, b, c = 3, 2, 1
print(a > b > c) # True: (a > b) and (b > c)

# contam como FALSO em um if:
# 0 0.0 "" [] {} None
# contam como VERDADEIRO:
# "0" [0] " "
```

Linguagem	O que o <code>if</code> aceita
C, Python, JavaScript	qualquer valor (0, vazio e nulo contam como falso)
Java, C#, Go, Rust	somente booleano : <code>if (n)</code> com <code>n</code> inteiro nem compila

Uso correto da expressão respeitando parênteses e meios de comparação separando por expressões.

Avaliação em curto-circuito

Java - && faz curto-circuito, & não

```
// busca do livro (p. 316)
while (i < lista.length && lista[i] != chave)
    i++;           // && evita lista[lista.length]

String s = null;
s != null && s.length() > 0    // false
s != null & s.length() > 0    // NullPointerException
```

JavaScript - ?. e ??

```
const u = { nome: "Ana", endereco: null };
u.endereco?.cidade           // undefined, sem erro

const qtd = 0;
qtd || 10                    // 10  (o 0 se perdeu!)
qtd ?? 10                    // 0   (só troca null/undefined)
```

Cuidado com efeitos colaterais (p. 316)
Em (a > b) || (b++ / 3), o b++ só acontece quando a <= b. Em Python, 0 or "padrão" devolve "padrão": and e or devolvem um dos **operandos**.

Percebemos que ao utilizar o curto-circuito devemos prestar atenção no logica utilizado podendo muitas vezes nem ser executado se não utilizarmos de forma correta.

Sentenças de atribuição

C - a atribuição é uma expressão (p. 319-320)

```
sum = count = 0;           // múltiplos alvos
a = b + (c = d / b) - 1;    // exemplo do livro

while ((ch = getchar()) != EOF) { ... }

if (x = y) { ... }         // compila! atribui e testa y
```

Linguagem	Atribuição é expressão?
C, C++, JavaScript	sim: if (x = y) compila
Java, C#	sim, mas o if exige booleano
Python	não, exceto o operador morsa :=
Go	não: nem x++ é expressão
Rust	vale (): if x = 5 não compila

Python - atribuição múltipla (p. 320)

```
a, b = b, a                # troca sem temporária
if (n := len(dados)) > 5: ... # morsa (Python 3.8+)
```

Linguagens funcionais (p. 321)
val (ML) e let (F#, Rust) dão **nome** a um valor, que não muda.

Diversas atribuições de sentenças vemos que as linguagens podem ter algumas diferenças entre a forma de escrita de algumas expressões, como =, :=

Atribuição de modo misto

Java - só alargamento (p. 322)

```
double d = 3;           // ok: alargamento
// int i = 3.7;         // erro: estreitamento
int i = (int) 3.7;       // 3 (cast explícito)
long l = i;              // ok

i += 3.7;                // COMPILA: cast escondido
System.out.println(i);   // 6
```

C - coerção em qualquer direção

```
int i = 3.7;             // 3, em silêncio
char c = 300;            // 44 (só um aviso)
unsigned int u = -1;      // 4294967295
```

Armadilha clássica

`double taxa = acertos / total * 100;` com inteiros dá **0.0**: a divisão inteira acontece **antes** do alargamento para double.

Linguagens funcionais: não existe atribuição de modo misto, o java e c# só permitem alargamento de int para float

Seleção de dois caminhos: o "else pendente"

C - a endentação engana (p. 331)

```
int sum = 0, count = 5, result = -1;
if (sum == 0)
    if (count == 0)
        result = 0;
else
    result = 1;           // de qual if é este else?
printf("%d\n", result);  // 1
```

Regra de Java, C, C++ e C#

O else pertence ao if **mais próximo** que ainda não tem else. A endentação é ignorada (exceto em Python e F#).

Chaves obrigatórias eliminam o problema: Perl, Go e Rust (p. 332).

Seletores que devolvem valor

```
let y = if x > 0 { x } else { 2 * x };           // Rust: if é uma expressão (como em F#, p. 334)
y = x if x > 0 else 2 * x                        # Python: expressão condicional (p. 306)
```

Sebesta 11ª ed. - 6R 2 1 - n 330-334 - exemplos/07 - selecion

```
1 #include <stdio.h>
2 int main()
3 {
4     int sum = 0, count = 5, result = -1;
5     if (sum == 0)
6         if (count == 0)
7             result = 0;
8     else
9         result = 1;
10    printf("%d\n", result);
11 }
```

A endentação deve ser descrita de forma mais visível pois pode ser ignorada pelo compilador em alguns casos.

Desvio incondicional: o goto

"goto fail" (Apple, 2014) - versão simplificada

```
int valida_certificado(void) {
    int err;
    if ((err = verifica_hash()) != 0)
        goto fail;
    goto fail;           // duplicada: desvia SEMPRE
    if ((err = verifica_assinatura()) != 0)
        goto fail;      // nunca é alcançada
fail:
    return err;          // 0 = "certificado válido"
}
```

Dijkstra (1968)

"Go To Statement Considered Harmful"

Quem tem goto hoje

C, C++, C#, Go. Java **não** tem (p. 353)

Gotos restritos

break, continue e return melhoram a legibilidade (p. 353)

Com chaves obrigatórias (Go, Rust) ou com gcc -Wall (aviso de endentação enganosa), o bug teria sido pego.

Comandos protegidos de Dijkstra

```
if x >= y -> max := x
[] y >= x -> max := y
fi
```

Exemplo do livro (p. 354): se $x == y$, qualquer ramo serve.

- Todas as **guardas** são avaliadas
- Várias verdadeiras: escolha **não determinística**
- Nenhuma verdadeira: erro (no if) ou fim do laço (no do ... od)
- Base do **CSP** de Hoare (1978) e do select de Go

Go · select, herdeiro dos comandos protegidos

```
a <- "a"      // os dois canais
b <- "b"      // estão prontos

select {      // Go sorteia uma guarda
case x := <-a:
    contagem[x]++
case x := <-b:
    contagem[x]++
}

// 1000 repetições:
// map[a:505 b:495] (muda a cada execução)
```

Exercício em duplas: encontre o bug (1/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

1 · JavaScript

```
const precos = [10, 20, 30];
let total = 0;
for (const p in precos)
    total += p;
console.log("Total: " + total);
```

2 · Python

```
def desconto(preco, d=None):
    d = d or 10 # padrão: 10%
    return preco * (100 - d) / 100

print(desconto(200, 0))
```

Exercício: encontre o bug (2/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

3 · Java

```
int dia = 2; String nome = "";
switch (dia) {
    case 1: nome = "domingo";
    case 2: nome = "segunda";
    case 3: nome = "terça";
    default: nome = "inválido";
}
System.out.println(nome);
```

4 · C

```
int saldo = 100, saque = 50;
if (saque > saldo);
    printf("saldo insuficiente\n");
printf("fim\n");
```

Exercício: encontre o bug (3/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

5 · Go

```
x := 10
if x > 5 {
    x := x * 2
    fmt.Println("dentro:", x)
}
fmt.Println("fora:", x)
```

6 · Java

```
int acertos = 7, total = 10;
double taxa = acertos / total * 100;
System.out.println(taxa + "%");
```

Síntese: armadilhas e como as linguagens ajudam

Tema	Armadilha	Como as linguagens ajudam
Precedência	<code>-2 ** 2, 1 + 2 + "3"</code>	parênteses; JS obriga <code>(-2) ** 2</code>
Operandos	<code>a + f()</code> com efeito colateral (C, Go)	Java fixa a ordem; funções puras
Conversões	coerção silenciosa (C, JS, PHP)	Go e Rust exigem conversão explícita; <code>==</code>
Booleanos	<code>0, "", []</code> são falsos; <code>a > b > c</code> em C	<code>if</code> só com booleano (Java, Go, Rust); <code>??</code>
Atribuição	<code>if (x = y)</code>	atribuição não é expressão (Python, Go, Rust)
Seleção	<code>switch</code> sem <code>break</code> ; <code>else</code> pendente	<code>-></code> , <code>match</code> exaustivo, chaves obrigatórias
Laços	<code>for...in</code> no JS; laço sem progresso	<code>for...of</code> , iteradores, <code>break</code> rotulado
Desvio	<code>goto</code> sem restrição	<code>break</code> , <code>continue</code> , <code>return</code> : <code>gotos</code> restritos

Atividades complementares (autopesquisa)

- **Expressões em Ruby e em Lisp** (§7.2.1.4-7.2.1.5, p. 305-306): `expressoes.rb`, `expressoes.lisp`
- **Expressões condicionais** (§7.2.1.6, p. 306) e **erros em expressões** (§7.4.3, p. 313)
- **Implementação da seleção múltipla**: tabelas de desvio (§8.2.2.3, p. 337-339)
- **Laços em linguagens funcionais**: contagem por recursão (§8.3.1.4, p. 345)
- **Iteradores e blocos de Ruby** (§8.3.4, p. 349-352)
- Leitura: Dijkstra, "Go To Statement Considered Harmful" (1968)

Entrega sugerida

Um mesmo exemplo com `switch` em C, `switch` -> em Java, `switch` em Go e `match` em Rust: compare "cair" entre casos, exaustividade e legibilidade.

Próxima aula

Subprogramas (Caps. 9 e 10)

Resumo deste bloco:

Nos capítulos 7 e 8, aprendemos sobre operadores, expressões e estruturas de seleção usadas nas linguagens de programação. O `switch` é uma estrutura usada para escolher uma ação de acordo com um determinado valor, mas funciona de maneiras diferentes em cada linguagem. Em C, usamos `switch`, `case` e normalmente `break`; em Java, versões mais novas permitem usar `->`, deixando o código mais simples; em Go, não é necessário usar `break` normalmente; e em Rust existe o `match`, que é parecido com o `switch`, mas permite trabalhar com diferentes padrões de forma mais completa.

Em Ruby, os operadores são considerados métodos, então uma expressão como `1 + 2` pode ser entendida como `1.+(2)`, e esses operadores podem até ser redefinidos. Já em Lisp, é usada a notação prefixa, em que o operador aparece antes dos valores, como `(+ 1 2)`, o que evita problemas de precedência. Também vimos as tabelas de desvio, que podem tornar um `switch` com muitos casos mais rápido do que uma sequência grande de `if`, pois o programa pode encontrar diretamente o caminho correspondente ao caso. Por fim, o texto de Dijkstra, de 1968, é uma leitura histórica importante sobre a forma de pensar e escrever programas de maneira mais organizada e correta. Na próxima aula, vamos estudar subprogramas, incluindo parâmetros, passagem por valor e por referência, fechamentos e a pilha de execução.